Problem 1.

Solution (1a)

To find the value of w^*, we differentiate the objective function with respect to w and set it to zero.

First, let's denote the object function as $J(w)$.

$$J(w) = \frac{1}{2}(\sum_{i=1}^{n}\left(y^{(i)} - wx^{(i)}\right)^2 + \prec w^2)$$

$$J(w) = \frac{1}{2}\sum_{i=1}^{n}\left(y^{(i)} - wx^{(i)}\right)^2 + \frac{1}{2}\prec w^2$$

$$\frac{dJ}{dw} = -\sum_{i=1}^{n} x^{(i)}\left(y^{(i)} - wx^{(i)}\right) + \prec w$$

Setting this derivative to zero, we have

$$-\sum_{i=1}^{n} x^{(i)}\left(y^{(i)} - wx^{(i)}\right) + \prec w = 0$$

Expanding and rearranging terms.

$$-\sum_{i=1}^{n} x^{(i)}y^{(i)} + w\sum_{i=1}^{n}\left(x^{(i)}\right)^2 + \prec w = 0$$

$$-\sum_{i=1}^{n} x^{(i)}y^{(i)} + w(\sum_{i=1}^{n}\left(x^{(i)}\right)^2 + \prec) = 0$$

$$w\left(\sum_{i=1}^{n}\left(x^{(i)}\right)^2 + \prec\right) = \sum_{i=1}^{n} x^{(i)}y^{(i)}$$

$$w = \frac{\sum_{i=1}^{n} x^{(i)}y^{(i)}}{\sum_{i=1}^{n}(x^{(i)})^2 + \prec}$$

So, the explicit formula for w^* is

$$w = \frac{\sum_{i=1}^{n} x^{(i)}y^{(i)}}{\sum_{i=1}^{n}(x^{(i)})^2 + \prec}$$

Solution (1b)

As $\prec$ goes from 0 to infinity, w^* changes. When $\prec$ is 0, the regularization term has no effect, so w^* is simply the ordinary least square solution or standard linear regression. Mathematically, $\prec = 0$: No regularization. w^* are larger parameters that fit training data well.

As λ increases, the regularization term penalizes larger values of w, causing w^* to decrease towards zero. This means that as λ increases, the model puts more emphasis on fitting the data with smaller weights, effectively reducing the impact of outliers, and makes the model more robust. Mathematically, $\lambda \to \infty$: Maximizes regularization, forces $w^* \to 0$. The close form of w is a monotonic function, therefore increase in λ makes the denominator larger and w shrinks to zero.

Problem 2.

Solution (2a) and (2b).

The link below provides the Scikit-learn LinearRegression() solutions of (2a) and (2b).

Untitled26.ipynb - Colaboratory (google.com)

```python
from sklearn.linear_model import LinearRegression
# Define the initial observations
X = [[1, 1]] * 100 + [[1, 0]]  # Initial observations: 100 with (1 keyboard, 1 mouse) and 1 with (1 keyboard, 0 mouse)
Y = [1] * 100 + [1]  # Corresponding target values (number of computers sold)

# Fit the linear regression model with initial data without intercept
reg = LinearRegression(fit_intercept=False).fit(X, Y)
# Get the coefficients for initial data
w1 = reg.coef_[0]  # Coefficient for k
w2 = reg.coef_[1]  # Coefficient for m
print("Optimal values of w1 and w2 with initial observations (without intercept):")
print("w1 =", w1)
print("w2 =", w2)
# Update the dataset with new observations
X.extend([[0, 1], [0, 1]])
Y.extend([1, 1])
# Fit the linear regression model with updated data without intercept
reg_new = LinearRegression(fit_intercept=False).fit(X, Y)
# Get the coefficients with updated data
w1_new = reg_new.coef_[0]  # Updated coefficient for k
w2_new = reg_new.coef_[1]  # Updated coefficient for m
print("\nOptimal values of w1 and w2 with new observations (without intercept):")
print("w1 =", w1_new)
print("w2 =", w2_new)
```



```
Optimal values of w1 and w2 with initial observations (without intercept):
w1 = 1.0000000000000004
w2 = 6.383202938381328e-17

Optimal values of w1 and w2 with new observations (without intercept):
w1 = 0.33774834437086143
w2 = 0.6688741721854303
```

NB: for initial observation $w_1 \approx 1 \; and \; w_2 \approx 0$.

Solution (2c)

This problem occurs because with the mostly homogeneous data (nearly all 1 keyboard, 1 mouse, 1 computer), the inputs are highly correlated. This causes instability in the fitted regression parameters. Even a small change in observations can cause the estimated w_1 and w_2 to change drastically, as we saw going from all 1s to adding 0s. In a real model, we would expect the true underlying parameters to be more stable representations of the relationships.

To check if a dataset may encounter this problem, we can:

- Assess the variability (variance) of each input feature - low variance indicates potential instability.
- Compute the correlation matrix between input features - high correlation will lead to unstable and unpredictable parameter estimates.
- Calculate the condition number of the input matrix X - higher values signify more sensitivity to data changes.
- Visually plot and examine the data - lacking variability in inputs can be spotted through plots.

By checking these metrics on a candidate dataset, we can detect possible issues with regression stability before even fitting a model. This allows us to preemptively handle datasets that may lead to jumping or unstable coefficient values. Taking remedial steps like adding regularization or removing highly correlated features can help mitigate these issues. Overall, inspecting the collinearity and variability in the inputs is key to avoiding wildly fluctuating parameter estimates.

Problem 3.

Solution (3a)

To compute the value of the decision function z at point $A(x_1 = 3, x_2 = 3)$, we can simply substitute the values of x_1 and x_2 into the given decision function:

$$z(x_1, x_2) = 4x_1 + 6x_2 - 24$$

Substituting $x_1 = 3$ and $x_2 = 3$, we get.

$$z(3,3) = 4(3) + 6(3) - 24$$

$$z(3,3) = 12 + 18 - 24$$

$$z(3,3) = 6$$

So, the value of the decision function z at the point A is 6.

Computing the probability of point, A being a positive example using the sigmoid function, we have.

$$p(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-6}}$$

$$p(y = 1|x) = \frac{1}{1 + 0.00248} \approx \frac{1}{1.00248} \approx 0.9975 \approx 0.998$$

Solution (3b)

To draw the decision boundary, we set the decision function $z(x_1, x_2) = 0$.

From the decision function $z(x_1, x_2) = 4x_1 + 6x_2 - 24$, we solve for x_2.

$$4x_1 + 6x_2 - 24 = 0$$

$$4x_1 + 6x_2 = 24$$

$$6x_2 = -4x_1 + 24$$

$$x_2 = \frac{-4x_1 + 24}{6}$$

$$x_2 = \frac{-2x_1}{3} + 4$$

When $x_1 = 0$, $x_2 = 4$ and $x_2 = 0$, $x_1 = 6$. The decision boundary is drawn as shown below.

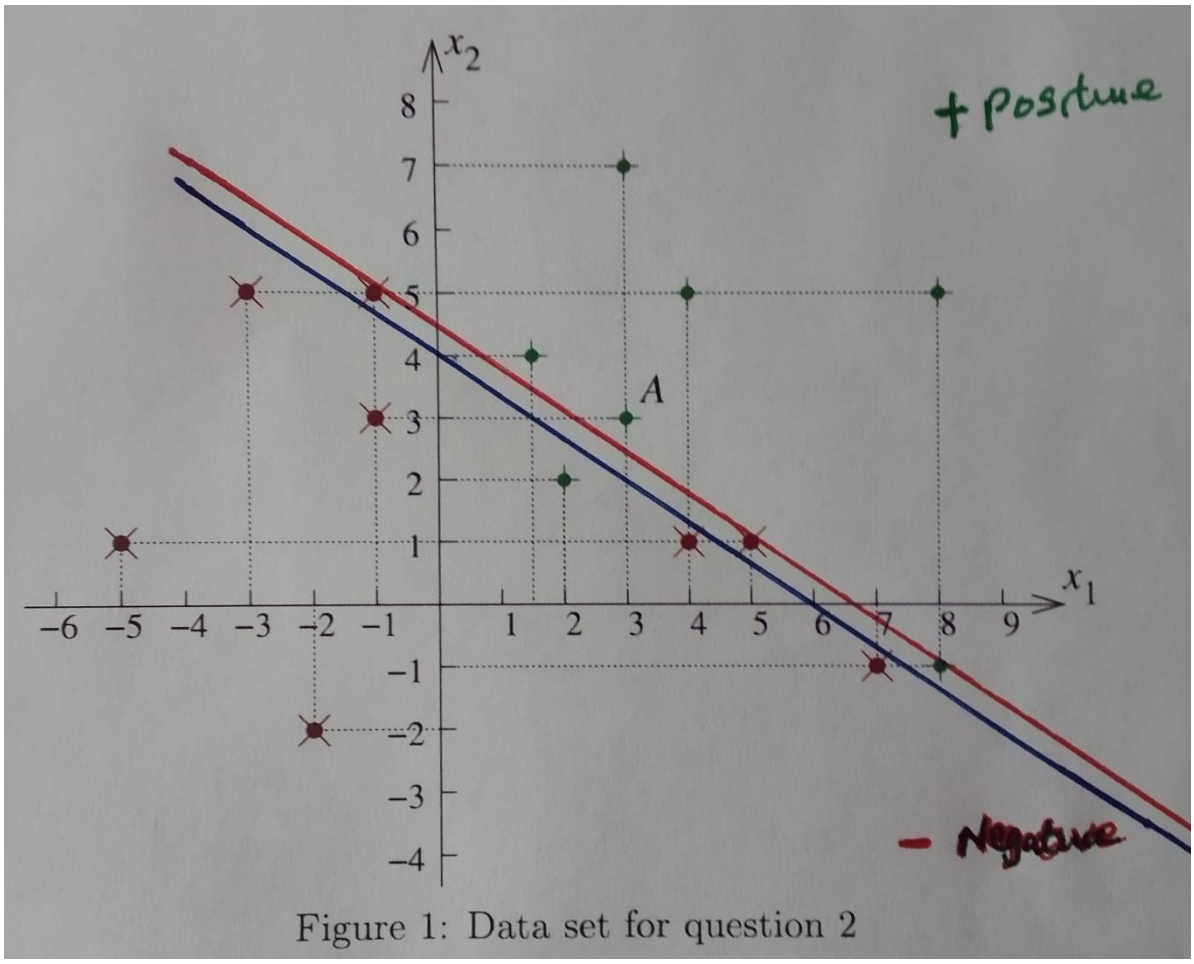

Figure 1: Data set for question 2

Figure 1 shows the decision boundary with a blue line, and it passes through the coordinates (0, 4) and (6, 0). The (+) positive region is represented with a function $f(x_1, x_2) > 0$ and the (-) negative region is represented with a function $f(x_1, x_2) < 0$, representing the graph upper and lower regions respectively. Also, the red line is the linear decision boundary that improves upon the accuracy. Figure 2 explicitly represents the blue line of the original decision boundary before improvement upon the accuracy was applied.

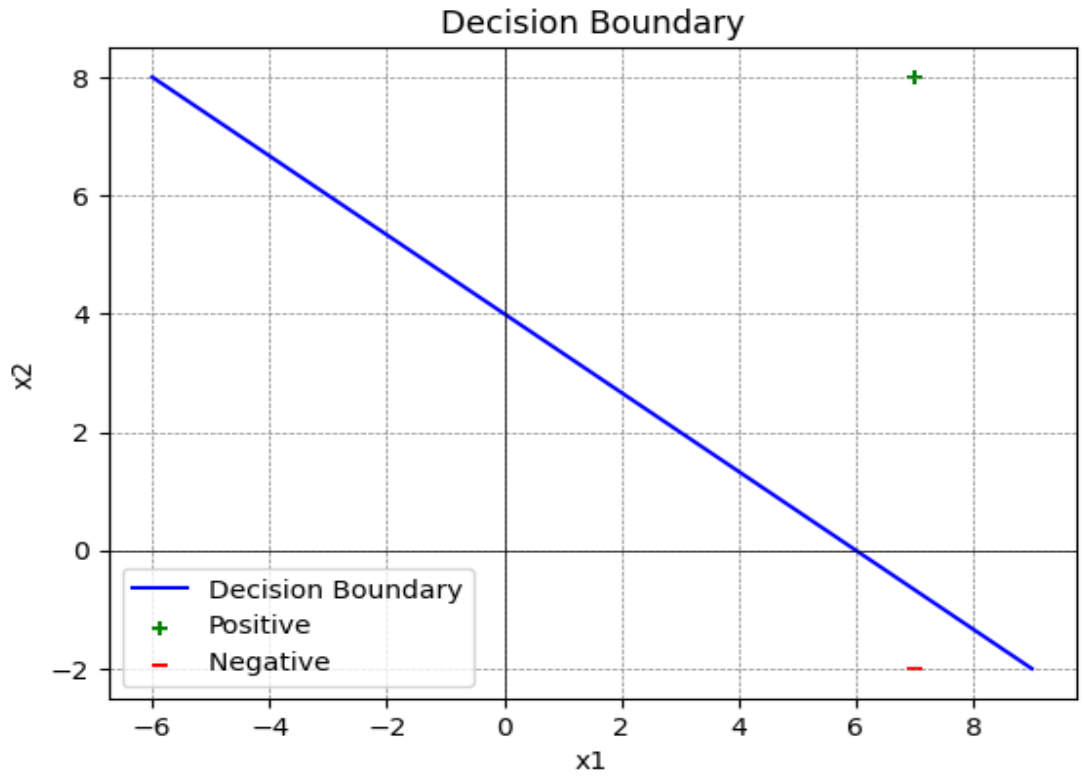

Figure 2. Decision boundary

Figure 2 shows the decision boundary with a blue line, and it passes through the coordinates (0, 4) and (6, 0). The positive region is represented with a function $f(x_1, x_2) > 0$ and the negative region is represented with a function $f(x_1, x_2) < 0$, indicated with (+) and (-) respectively.

Solution (3c)

From the decision boundary we can see that.

TP = 6	FN = 1
FP = 2	TN = 6

Where TP = True Positive; TN = True Negative; FP = False Negative; FN = False Negative.

Accuracy of the linear classifier is computed as follows.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Accuracy = \frac{6 + 6}{6 + 6 + 2 + 1} = \frac{12}{15} = 0.8 = 80\%$$

Solution (3d)

To improve upon the accuracy of a linear decision function, we typically aim to find a decision boundary that better separates the classes in the dataset. One way to achieve this is by using a linear decision function with coefficients that are optimized to minimize misclassifications or maximize a chosen performance metric.

Coordinates of the linear decision boundary that improves upon the accuracy are $(x_1, x_2) = (6.6\ , 0)$ and $(x_1, x_2) = (0,\ 4.5)$.

Applying the equation of a line we have.

$$\frac{x_1}{6.6} + \frac{x_2}{4.5} = 1$$

$$\frac{4.5x_1 + 6.6x_2}{29.7} = 1$$

$$4.5x_1 + 6.6x_2 = 29.7$$

$$4.5x_1 + 6.6x_2 - 29.7 = 0$$

From the given graph $z'(x_1, x_2) = 4.5x_1, +6.6x_2 - 29.7 = 0$. Here the previous line is shifted up by 0.5 units.

Considering the red linear line on the graph that improves upon the accuracy, we can see that the accuracy improved with additional 6.7% resulting to the 86.7% improved accuracy instead of 80% of the original accuracy.

TP = 5	FN = 2
FP = 0	TN = 8

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Accuracy = \frac{5 + 8}{5 + 8 + 0 + 2} = \frac{13}{15} = 0.867 = 86.7\%$$

Problem 4.

Solution (4a)

1. Too much variance in the training data. If there is high variance, the model may learn to fit the noise or random fluctuations in the data rather than capturing the underlying patterns, leading to overfitting.
2. Too much noise in the training data. Noise refers to random fluctuations or errors in the training data that do not reflect the true underlying relationship between the features and the target variable. If the training data contains a significant amount of noise, the model may learn to fit the noise rather than the underlying pattern, leading to overfitting.
3. Insufficient regularization. Insufficient regularization techniques, such as L1 or L2 regularization, may fail to penalize complex models, allowing them to overfit the data.
4. Limited training data size. When the training dataset is small, the model may not be able to generalize well to unseen data, as it hasn't captured enough information about the underlying patterns in the data.
5. High model complexity. If the model is too complex relative to the amount of training data available, it may capture noise or random fluctuations in the data instead of the underlying patterns. This often occurs with models that have too many parameters or features relative to the dataset size.

Solution (4b)

Overfitting can be avoided through these decision tree models, and regression models approaches.

1. Pre-pruning. Stop growing the tree at some point during construction when it is determined that there is not enough data to make reliable choices.
2. Post-pruning. Grow the tree and then remove nodes that seem not to have sufficient evidence. Pruning the decision tree to remove branches that do not significantly improve performance on the validation set, reducing the model complexity.
3. Increasing the amount of training data. One of the main causes of overfitting is when the model learns to memorize the training data rather than generalize from it. By providing more diverse and representative data for training, the model is less likely to memorize specific examples and instead learns more general patterns. This can help reduce overfitting because the model becomes better at generalizing to unseen data.
4. Using methods to control model complexity. Overly complex models are more prone to overfitting because they can learn intricate details of the training data that may not generalize well. Regularization (e.g., L1 or L2 regularization) technique is employed in logistic regression. This method constrains the model's capacity, making it less likely to overfit by discouraging it from fitting noise in the training data. The logistic regression adds a regularization term to the loss function to constrain model complexity. This is represented by the equation below.

$$L_{regularized}(w) = L(w) + \lambda \sum_{j=1}^{d} w_j^2$$

Solution (4c)

Accuracy can be a bad metric to evaluate a classifier, especially in imbalanced datasets, where one class significantly outnumbers the other. In such cases, a classifier that always predicts the majority class can achieve high accuracy but fails to detect instances of the minority class. Additionally, accuracy does not provide insights into the types of errors made by the classifier. For example, in a 2-class problem with 9990 examples of class 0 and only 10 examples of class1, a model that predicts all instances as class 0 would achieve a high accuracy of 99.9%. However, this accuracy is misleading because the model fails to detect any class 1 examples.

Other metrics that can be used include.

1. Precision. The proportion of true positive predictions among all positive predictions. It measures the model's ability to correctly identify positive instances.

$$Precision\ (p) = \frac{TP}{TP + FP}$$

2. Recall: The proportion of true positive predictions among all actual positive instances. It measures the model's ability to capture all positive instances.

$$Recall\ (r) = \frac{TP}{TP + FN}$$

3. F-measure (F1-score). The harmonic means of precision and recall, providing a balanced measure of the classifier's performance.

$$F - measure = 2 * (Precision * recall)/(precision + recall)$$

$$F - measure = \frac{2TP}{2TP + FP + FN}$$

Solution (4d).

M is the order of the polynomial.

1. In Figure 2(b), when we fit a linear model $(M = 1)$ to the observations generated from the $\sin(2\pi x)$ function, the model is not a good fit. In this case, $(M = 1)$ is a first-order polynomial and is too simple to accurately represent the function $\sin(2\pi x)$, leading to a poor fit and inadequate representation of the data. It is underfitting because a linear model cannot capture the non-linear relationship between the input variable x and the target variable y.

2. In Figure 2(c), when we fit a higher-order polynomial $(M = 9)$ to the observations, we obtain an excellent fit to the training data. This suggests a strong alignment to the training data and not to the validation or test dataset and does not imply the model will generalize well to unseen data. It may be seen as overfitting because the polynomial model with high complexity is fitting the noise in the data instead of the underlying pattern, resulting in poor generalization to new data.